

Experiment 9:

Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite). Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

Code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define BUFFER_SIZE 1024

void copy_using_system_calls(const char *source, const char *destination)
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytesRead, bytesWritten;

    src = open(source, O_RDONLY);

    if (src == -1)
    {
        perror("Error opening source file");
        return;
    }

    dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest == -1)
    {
        perror("Error opening destination file");
        close(src);
        return;
    }

    while ((bytesRead = read(src, buffer, BUFFER_SIZE)) > 0)
    {
        bytesWritten = write(dest, buffer, bytesRead);

        if (bytesWritten != bytesRead)
        {
            perror("Error writing file");
            break;
        }
    }
}
```

```

    }

    close(src);
    close(dest);

    printf("File copied using system calls.\n");
}

void copy_using_library(const char *source, const char *destination)
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytesRead;

    src = fopen(source, "r");
    dest = fopen(destination, "w");

    if (src == NULL || dest == NULL)
    {
        perror("Error opening file");
        return;
    }

    while ((bytesRead = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
    {
        fwrite(buffer, 1, bytesRead, dest);
    }

    fclose(src);
    fclose(dest);

    printf("File copied using standard library functions.\n");
}

void demonstrate_redirection()
{
    int inputFile, outputFile;

    inputFile = open("input.txt", O_RDONLY);

    if (inputFile == -1)
    {
        perror("Error opening input.txt");
        return;
    }

    outputFile = open("output.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

    if (outputFile == -1)
    {

```

```

        perror("Error opening output.txt");
        close(inputFile);
        return;
    }

    dup2(inputFile, STDIN_FILENO);
    dup2(outputFile, STDOUT_FILENO);

    close(inputFile);
    close(outputFile);

    printf("This output has been redirected to output.txt\n");
}

int main()
{
    int choice;

    printf("==== EXPERIMENT 9 =====\n");
    printf("1. File Copy\n");
    printf("2. I/O Redirection using dup2()\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    if (choice == 1)
    {
        copy_using_system_calls("source.txt",
                                "destination.txt");

        copy_using_library("source.txt",
                            "destination_library.txt");

        printf("Both file-copy methods completed.\n");
    }
    else if (choice == 2)
    {
        demonstrate_redirection();
    }
    else
    {
        printf("Invalid choice.\n");
    }

    return 0;
}

```

Output:

```
adarohan18@Rohith:~$ touch source.txt
adarohan18@Rohith:~$ nano source.txt
adarohan18@Rohith:~$ cat source.txt
Hello, I am ADA ROHAN FROM OS S6
This is a sample source file.
This file is used for Experiment 9.
We are demonstrating file copying using system calls.
adarohan18@Rohith:~$ gcc Practical25.c -o Practical25
adarohan18@Rohith:~$ ./Practical25
===== EXPERIMENT 9 =====
1. File Copy
2. I/O Redirection using dup2()
Enter your choice: 1
File copied using system calls.
File copied using standard library functions.
Both file-copy methods completed.
adarohan18@Rohith:~$ |
```

Experiment 10:

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

Code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <stdlib.h>
#include <string.h>

#define SIZE 100

void inode_demo()
{
    FILE *file;
    struct stat st;

    file = fopen("original.txt", "w");

    if (file == NULL)
    {
        perror("Error creating file");
        return;
    }
}
```

```

}

fprintf(file, "Hello from Experiment 10\n");
fclose(file);

if (stat("original.txt", &st) == -1)
{
    perror("stat");
    return;
}

printf("\nOriginal file inode: %lu\n",
        (unsigned long)st.st_ino);

if (link("original.txt", "hardlink.txt") == -1)
{
    perror("Hard link");
}
else
{
    printf("Hard link created.\n");
}

if (symlink("original.txt", "symlink.txt") == -1)
{
    perror("Symbolic link");
}
else
{
    printf("Symbolic link created.\n");
}

if (stat("hardlink.txt", &st) == 0)
{
    printf("Hard link inode: %lu\n",
            (unsigned long)st.st_ino);
}

if (lstat("symlink.txt", &st) == 0)
{
    printf("Symbolic link inode: %lu\n",
            (unsigned long)st.st_ino);
}
}

void mmap_demo()
{
    int fd;

```

```

struct stat st;
char *data;

fd = open("mmapfile.txt",
          O_RDWR | O_CREAT,
          0644);

if (fd == -1)
{
    perror("open");
    return;
}

ftruncate(fd, SIZE);

if (fstat(fd, &st) == -1)
{
    perror("fstat");
    close(fd);
    return;
}

data = mmap(NULL,
            SIZE,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd,
            0);

if (data == MAP_FAILED)
{
    perror("mmap");
    close(fd);
    return;
}

strcpy(data, "Hello from memory mapped file I/O!");

printf("\nData written using mmap():\n");
printf("%s\n", data);

msync(data, SIZE, MS_SYNC);

munmap(data, SIZE);
close(fd);

printf("Memory-mapped file operation completed.\n");
}

```

```

int main()
{
    int choice;

    printf("==== EXPERIMENT 10 =====\n");
    printf("1. Inode and Link Demonstration\n");
    printf("2. Memory-Mapped File I/O\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    if (choice == 1)
    {
        inode_demo();
    }
    else if (choice == 2)
    {
        mmap_demo();
    }
    else
    {
        printf("Invalid choice.\n");
    }

    return 0;
}

```

Output:

```

adarohan18@Rohith:~$ nano Practical26.c
adarohan18@Rohith:~$ gcc Practical26.c -o Practical26
adarohan18@Rohith:~$ ./Practical26
==== EXPERIMENT 10 =====
1. Inode and Link Demonstration
2. Memory-Mapped File I/O
Enter your choice: 1

Original file inode: 26368
Hard link created.
Symbolic link created.
Hard link inode: 26368
Symbolic link inode: 26369
adarohan18@Rohith:~$ |

```

Experiment 11:

Develop a multithreaded counter application using POSIX threads (`pthread_create`, `pthread_join`). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

Code:

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

pthread_mutex_t mutex;

void *increment_without_mutex(void *arg)
{
    int i;

    for (i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    return NULL;
}

void *increment_with_mutex(void *arg)
{
    int i;

    for (i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&mutex);

        counter++;

        pthread_mutex_unlock(&mutex);
    }

    return NULL;
}

void run_race_condition()
{

```

```

pthread_t threads[NUM_THREADS];
int i;

counter = 0;

for (i = 0; i < NUM_THREADS; i++)
{
    pthread_create(&threads[i],
                  NULL,
                  increment_without_mutex,
                  NULL);
}

for (i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("\n--- Race Condition ---\n");
printf("Expected Counter: %d\n",
       NUM_THREADS * ITERATIONS);

printf("Actual Counter: %d\n",
       counter);
}

void run_with_mutex()
{
    pthread_t threads[NUM_THREADS];
    int i;

    counter = 0;

    pthread_mutex_init(&mutex, NULL);

    for (i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i],
                      NULL,
                      increment_with_mutex,
                      NULL);
    }

    for (i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("\n--- With Mutex ---\n");
    printf("Expected Counter: %d\n",
          NUM_THREADS * ITERATIONS);
}

```

```

        printf("Actual Counter: %d\n",
               counter);

        pthread_mutex_destroy(&mutex);
    }

    int main()
    {
        printf("==== EXPERIMENT 11 =====\n");

        run_race_condition();

        run_with_mutex();

        return 0;
    }

```

Output:

```

adarohan18@Rohith:~$ nano Practical27.c
adarohan18@Rohith:~$ gcc Practical27.c -o Practical27
adarohan18@Rohith:~$ ./Practical27
==== EXPERIMENT 11 =====

--- Race Condition ---
Expected Counter: 400000
Actual Counter: 146314

--- With Mutex ---
Expected Counter: 400000
Actual Counter: 400000
adarohan18@Rohith:~$ |

```

Experiment 12:

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Code:

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 10

```

```

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t buffer_mutex;

pthread_mutex_t resource1;
pthread_mutex_t resource2;

/* ----- PRODUCER ----- */

void *producer(void *arg)
{
    int i;

    for (i = 0; i < ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&buffer_mutex);

        buffer[in] = i;

        printf("Produced: %d\n", i);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&buffer_mutex);

        sem_post(&full);
    }

    return NULL;
}

/* ----- CONSUMER ----- */

void *consumer(void *arg)
{
    int i;
    int item;

    for (i = 0; i < ITEMS; i++)
    {

```

```

        sem_wait(&full);

        pthread_mutex_lock(&buffer_mutex);

        item = buffer[out];

        printf("Consumed: %d\n", item);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&buffer_mutex);

        sem_post(&empty);
    }

    return NULL;
}

/* ----- PRODUCER-CONSUMER ----- */

void producer_consumer()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&buffer_mutex, NULL);

    pthread_create(&producer_thread,
                  NULL,
                  producer,
                  NULL);

    pthread_create(&consumer_thread,
                  NULL,
                  consumer,
                  NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);

    pthread_mutex_destroy(&buffer_mutex);

```

```

    printf("Producer-Consumer completed.\n");
}

/* ----- DEADLOCK PREVENTION ----- */

void *thread1(void *arg)
{
    pthread_mutex_lock(&resource1);

    printf("Thread 1 acquired Resource 1\n");

    pthread_mutex_lock(&resource2);

    printf("Thread 1 acquired Resource 2\n");

    printf("Thread 1 is working...\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    /*
     * Resource ordering:
     * Always acquire Resource 1 first,
     * then Resource 2.
     */

    pthread_mutex_lock(&resource1);

    printf("Thread 2 acquired Resource 1\n");

    pthread_mutex_lock(&resource2);

    printf("Thread 2 acquired Resource 2\n");

    printf("Thread 2 is working...\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void deadlock_prevention()

```

```

{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1,
                  NULL,
                  thread1,
                  NULL);

    pthread_create(&t2,
                  NULL,
                  thread2,
                  NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Deadlock prevented using resource ordering.\n");
}

/* ----- MAIN ----- */

int main()
{
    printf("===== EXPERIMENT 12 =====\n");

    printf("\n--- Producer Consumer ---\n");
    producer_consumer();

    printf("\n--- Deadlock Prevention ---\n");
    deadlock_prevention();

    return 0;
}

```

Output:

```
adarohan18@Rohith:~$ nano Practical28.c
adarohan18@Rohith:~$ gcc Practical28.c -o Practical28
adarohan18@Rohith:~$ ./Practical28.c
-bash: ./Practical28.c: Permission denied
adarohan18@Rohith:~$ ./Practical28
===== EXPERIMENT 12 =====

--- Producer Consumer ---
Produced: 0
Consumed: 0
Produced: 1
Produced: 2
Consumed: 1
Consumed: 2
Produced: 3
Produced: 4
Produced: 5
Produced: 6
Produced: 7
Consumed: 3
Consumed: 4
Consumed: 5
Consumed: 6
Consumed: 7
Produced: 8
Produced: 9
Consumed: 8
Consumed: 9
Producer-Consumer completed.

--- Deadlock Prevention ---
Thread 1 acquired Resource 1
Thread 1 acquired Resource 2
Thread 1 is working...
Thread 2 acquired Resource 1
Thread 2 acquired Resource 2
Thread 2 is working...
Deadlock prevented using resource ordering.
adarohan18@Rohith:~$ |
```